

Name : Narendra Kumar S

Date 25-08-2025

Task

- Create a backup commit of the existing book and sales details project.
- Move the queries of the book store from the controller into the Service using Dependency injection
- (optional) Once the architecture is changed to service then implement the repo pattern also

Solution :

Controller Class :

```
using Day12api.Context;
using Day12api.Model;
using Day12api.Service;
using Day12api.Services;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Microsoft.Identity.Client;
using System;
using System.Collections.Generic;
using System.Linq;

namespace Day12api.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class BookController : ControllerBase
    {
        private readonly IBookService _bookService;

        public BookController(IBookService bookService)
        {
            _bookService = bookService;
        }

        [HttpGet("GetAllBooks")]
        public ActionResult<IEnumerable<Book>> GetAllBooks()
        {
            return Ok(_bookService.GetAllBooks());
        }
    }
}
```

```

[HttpPost("AddBook")]
public IActionResult AddBook(Book book)
{
    return Ok(_bookService.AddBook(book));
}

[HttpDelete("DeleteBook/{id}")]
public IActionResult DeleteBook(int id)
{
    var result = _bookService.DeleteBook(id);
    return result == "Book deleted successfully" ? Ok(result) : NotFound(result);
}

[HttpPost("AddAuthor")]
public IActionResult AddAuthor(Author author)
{
    return Ok(_bookService.AddAuthor(author));
}

[HttpGet("FetchAllNewBook")]
public IActionResult GetAllNewBooks()
{
    return Ok(_bookService.GetAllNewBooks());
}

[HttpPost("AddSalesInfo")]
public IActionResult AddSalesInfo(SalesEntry entry)
{
    var result = _bookService.AddSalesInfo(entry);
    return result == "Sales information added successfully" ? Ok(result) : NotFound(result);
}

[HttpPost("AddNewBook")]
public IActionResult AddNewBook(BookAndAuthorEntry entry)
{
    var result = _bookService.AddNewBook(entry);
    return result == "New book added successfully" ? Ok(result) : NotFound(result);
}
}
}

```

User Controller class:

```
using Day12api.Context;
using Day12api.Model;
using Day12api.Security;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity.Data;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Security.Claims;
using System.Security.Cryptography;
using System.Text;
using WebApi.Security;

namespace Day12api.Controllers
{
    public class UserController : Controller
    {
        MyAppDbContext appDbContext;
        Microsoft.Extensions.Options.IOptions<JWTOption> options;

        public UserController(MyAppDbContext ctxt,
Microsoft.Extensions.Options.IOptions<JWTOption> _jwtOption)
        {
            appDbContext = ctxt;
            options = _jwtOption;
        }

        [HttpPost("Login")]
        public IActionResult Login(Model.LoginRequest loginreq)
        {
            var name = loginreq.Username;
            var hashedPassword = GetHashPassword(loginreq.Password);

            var is_user_available = appDbContext.Users
                .Where(x => x.Username == name && x.Password == hashedPassword)
                .Count();

            if (is_user_available == 0)
            {
                return Unauthorized("Invalid username or password");
            }
        }
    }
}
```

```

else
{
    var claims = new List<Claim>
    {
        new(ClaimTypes.Name, name),
        new Claim("Password", hashedPassword)
    };

    var token = JwtService.createJwtToken(options.Value, claims);
    return Ok(new { Token = token });
}
}

```

```

[HttpPost("AddUser")]
public IActionResult AddUser(UserDTO user)
{
    user.Password = GetHashPassword(user.Password);
    appDbContext.Users.Add(user);
    appDbContext.SaveChanges();
    return Ok("User added successfully");
}

```

```

[Authorize]
[HttpGet("GetAllUsers")]
public IActionResult GetAllUsers()
{
    var users = appDbContext.Users.ToList();
    return Ok(users);
}

```

```

[HttpGet("GetAllUsers_v2")]
public IActionResult GetAllUsers_v2()
{
    var users = appDbContext.Users
        .Select(x => new { x.Username, x.Email })
        .ToList();

    return Ok(users);
}

```

```

private string GetHashPassword(string password)
{
    using (SHA256 sha = SHA256.Create())
    {
        byte[] passBytes = Encoding.UTF8.GetBytes(password);

```

```

        byte[] hashBytes = sha.ComputeHash(passBytes);
        StringBuilder builder = new StringBuilder();
        for (int i = 0; i < hashBytes.Length; i++)
        {
            builder.Append(hashBytes[i].ToString("x2"));
        }
        return builder.ToString();
    }
}
}
}

```

Service :

Interface :

```

using Day12api.Model;

namespace Day12api.Service
{
    public interface IBookService
    {
        IEnumerable<Book> GetAllBooks();
        string AddBook(Book book);
        string DeleteBook(int id);
        string AddAuthor(Author author);
        List<NewBook> GetAllNewBooks();
        string AddSalesInfo(SalesEntry entry);
        string AddNewBook(BookAndAuthorEntry entry);
    }
}

```

Service Class :

```

using Day12api.Model;
using Day12api.Repositories;
using Day12api.Services;

namespace Day12api.Service
{
    public class BookService : IBookService
    {
        private readonly IBookRepository _repository;
    }
}

```

```

public BookService(IBookRepository repository)
{
    _repository = repository;
}

public IEnumerable<Book> GetAllBooks()
{
    return _repository.GetAllBooks();
}

public string AddBook(Book book)
{
    _repository.AddBook(book);
    return "Book added successfully";
}

public string DeleteBook(int id)
{
    var result = _repository.DeleteBook(id);
    return result ? "Book deleted successfully" : "Book not found";
}

public string AddAuthor(Author author)
{
    _repository.AddAuthor(author);
    return "Author added successfully";
}

public List<NewBook> GetAllNewBooks()
{
    return _repository.GetAllNewBooks();
}

public string AddSalesInfo(SalesEntry entry)
{
    var book = _repository.GetBookByTitle(entry.Book_name);
    if (book == null)
        return "Invalid book name";

    var sales = new Sales
    {
        BookId = book.NewBookId,
        num_of_copies = entry.num_of_copies,
    }
}

```

```

        Year = entry.Year
    };

    _repository.AddSales(sales);
    return "Sales information added successfully";
}

public string AddNewBook(BookAndAuthorEntry entry)
{
    var author = _repository.GetAuthorByName(entry.AuthorName);
    if (author == null)
        return "Invalid author name";

    var book = new NewBook
    {
        Title = entry.BookTitle,
        AuthorId = author.AuthorId,
        price = entry.Price
    };

    _repository.AddNewBook(book);
    return "New book added successfully";
}
}
}

```

Output :

Swagger UI

localhost:5211/swagger/index.html

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5211/Book/AddBook' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 0,
    "title": "Harry",
    "author": "Jack",
    "publishDate": "2025-08-25T17:02:34.430Z",
    "genre": "Jolly",
    "price": 2500
  }'
```

Request URL

http://localhost:5211/Book/AddBook

Server response

Code	Details
200	Response body Book added successfully Response headers content-type: text/plain; charset=utf-8 date: Mon, 25 Aug 2025 17:02:58 GMT server: Kestrel transfer-encoding: chunked

Adding author :

localhost:5211/swagger/index.html

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5211/Book/AddAuthor' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "authorId": 0,
    "authorName": "Jack Dao"
  }'
```

Request URL

http://localhost:5211/Book/AddAuthor

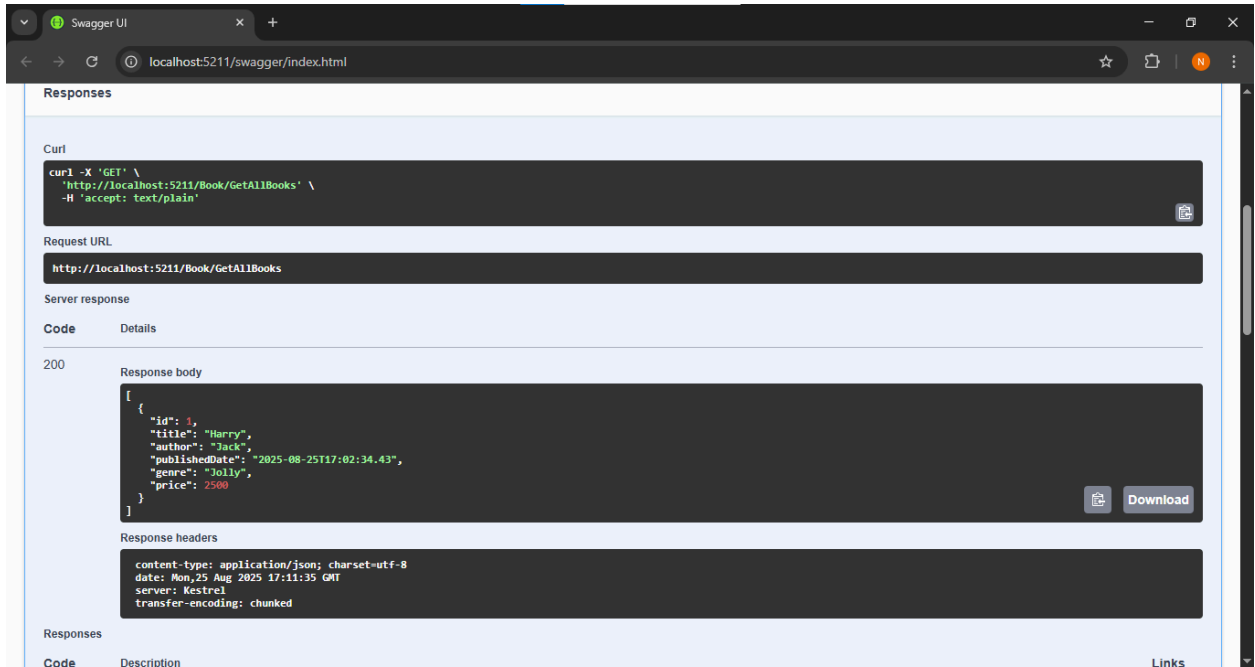
Server response

Code	Details
200	Response body Author added successfully Response headers content-type: text/plain; charset=utf-8 date: Mon, 25 Aug 2025 17:08:32 GMT server: Kestrel transfer-encoding: chunked

Responses

Code	Description	Link
------	-------------	------

Get all books :



A screenshot of the Swagger UI interface in a web browser. The browser tab is titled "Swagger UI" and the address bar shows "localhost:5211/swagger/index.html". The main content area is titled "Responses" and shows details for a GET request to "http://localhost:5211/Book/GetAllBooks". The request is shown in a dark box with the following curl command:

```
curl -X 'GET' \
  'http://localhost:5211/Book/GetAllBooks' \
  -H 'accept: text/plain'
```

 The response is a 200 status code with a JSON body:

```
{
  "id": 1,
  "title": "Harry",
  "author": "Jack",
  "publishedDate": "2025-08-25T17:02:34.43",
  "genre": "Jolly",
  "price": 2500
}
```

 The response headers are:

```
content-type: application/json; charset=utf-8
date: Mon, 25 Aug 2025 17:11:35 GMT
server: Kestrel
transfer-encoding: chunked
```

 At the bottom, there is a table with columns "Code" and "Description", and a "Links" section.

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5211/Book/GetAllBooks' \
  -H 'accept: text/plain'
```

Request URL

```
http://localhost:5211/Book/GetAllBooks
```

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "title": "Harry",
  "author": "Jack",
  "publishedDate": "2025-08-25T17:02:34.43",
  "genre": "Jolly",
  "price": 2500
}
```

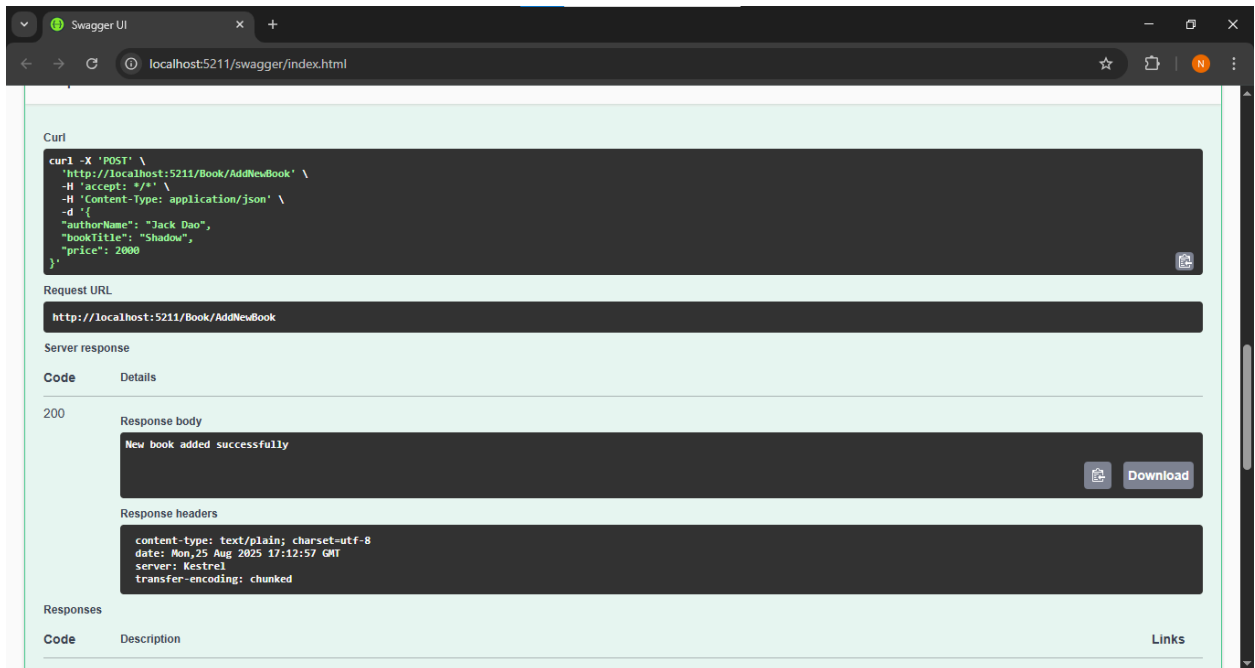
Response headers

```
content-type: application/json; charset=utf-8
date: Mon, 25 Aug 2025 17:11:35 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code Description Links

Add new Book :



A screenshot of the Swagger UI interface in a web browser. The browser tab is titled "Swagger UI" and the address bar shows "localhost:5211/swagger/index.html". The main content area is titled "Responses" and shows details for a POST request to "http://localhost:5211/Book/AddNewBook". The request is shown in a dark box with the following curl command:

```
curl -X 'POST' \
  'http://localhost:5211/Book/AddNewBook' \
  -H 'accept: */*' \
  -H 'Content-type: application/json' \
  -d '{
    "authorName": "Jack Dao",
    "bookTitle": "Shadow",
    "price": 2000
  }'
```

 The response is a 200 status code with a text body:

```
New book added successfully
```

 The response headers are:

```
content-type: text/plain; charset=utf-8
date: Mon, 25 Aug 2025 17:12:57 GMT
server: Kestrel
transfer-encoding: chunked
```

 At the bottom, there is a table with columns "Code" and "Description", and a "Links" section.

Curl

```
curl -X 'POST' \
  'http://localhost:5211/Book/AddNewBook' \
  -H 'accept: */*' \
  -H 'Content-type: application/json' \
  -d '{
    "authorName": "Jack Dao",
    "bookTitle": "Shadow",
    "price": 2000
  }'
```

Request URL

```
http://localhost:5211/Book/AddNewBook
```

Server response

Code Details

200

Response body

```
New book added successfully
```

Response headers

```
content-type: text/plain; charset=utf-8
date: Mon, 25 Aug 2025 17:12:57 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code Description Links

Fetch All New Book:

The image shows a Swagger UI interface in a web browser. The browser's address bar displays 'localhost:5211/swagger/index.html'. The interface is divided into several sections:

- Curl:** A text box containing the command:

```
curl -X 'GET' \
  'http://localhost:5211/Book/FetchAllNewBook' \
  -H 'accept: */*'
```
- Request URL:** A text box containing the URL: `http://localhost:5211/Book/FetchAllNewBook`
- Server response:** A section with two tabs: 'Code' (selected) and 'Details'.
- Code:** Displays the HTTP status code '200'.
- Response body:** A text box showing the JSON response:

```
{
  "newBookId": 1,
  "title": "Shadow",
  "price": 2000,
  "authorId": 1,
  "author": {
    "authorId": 1,
    "authorName": "Jack Dao"
  }
}
```

 To the right of the JSON is a 'Download' button.
- Response headers:** A text box showing the headers:

```
content-type: application/json; charset=utf-8
date: Mon, 25 Aug 2025 17:13:22 GMT
server: Kestrel
transfer-encoding: chunked
```
- Responses:** A section at the bottom, currently empty.